

Programación I

Prof. Carlos A. Benítez B.



Material de apoyo pedagógico

Comunitaria de Caacupé, 2023

Arrays, arreglos o vectores en C++

Presentación del tema.

El tema que vamos a abordar en la clase de hoy es arrays, arreglos o vectores en C++. Los arrays son estructuras de datos que permiten almacenar una colección de elementos del mismo tipo. Son muy útiles para trabajar con grandes conjuntos de datos y para llevar a cabo operaciones repetitivas sobre ellos.

En C++, los arrays se definen mediante una serie de elementos que se almacenan en una sola variable. Para acceder a los elementos individuales del array, se utilizan índices numéricos.

Los arrays en C++ pueden ser de cualquier tipo de datos, como enteros, flotantes, caracteres, estructuras, etc. Son muy flexibles y se pueden utilizar en una amplia variedad de aplicaciones, desde procesamiento de datos hasta aplicaciones gráficas.

Objetivos de la clase:

El objetivo principal de esta clase es aprender a trabajar con arrays en C++.

Los objetivos específicos son:

- ✓ Comprender el uso, declaración y sintaxis de los vectores en C++.
- ✓ Conocer cómo declarar e inicializar un array o vector en C++.
- ✓ Comprender las particularidades de los arrays o vectores en C++ y aprender a recorrerlos y obtener el valor de una casilla específica.
- ✓ Adquirir habilidades prácticas en el manejo de arrays en C++, a través de ejemplos y ejercicios prácticos.

Con estos objetivos, al final de la clase los estudiantes deberán ser capaces de utilizar los arrays en C++ de manera efectiva para manipular datos y realizar operaciones repetitivas de manera eficiente.

Declarar e inicializar un array unidimensional o vector en C++.

1. Arrays unidimensionales de tipo int:

```
int myArray[] = {1, 2, 3, 4, 5}; // Inicialización explícita
```

```
int anotherArray[5]; // Declaración con tamaño fijo
```

2. Arrays unidimensionales de tipo float:

```
float myArray[] = {1.5, 2.3, 3.7, 4.0, 5.1}; // Inicialización explícita
```

```
float anotherArray[5]; // Declaración con tamaño fijo
```

3. Arrays unidimensionales de tipo double:

```
double myArray[] = {1.5, 2.3, 3.7, 4.0, 5.1}; // Inicialización explícita
```

```
double anotherArray[5]; // Declaración con tamaño fijo
```

4. Arrays unidimensionales de tipo char:

```
char myArray[] = {'h', 'e', 'l', 'l', 'o'}; // Inicialización explícita
```

```
char anotherArray[6]; // Declaración con tamaño fijo
```

5. Arrays unidimensionales de tipo bool:

```
bool myArray[] = {true, false, true, true, false}; // Inicialización explícita
```

```
bool anotherArray[5]; // Declaración con tamaño fijo
```

6. Arrays unidimensionales de tipo short:

```
short myArray[] = {1, 2, 3, 4, 5}; // Inicialización explícita
```

```
short anotherArray[5]; // Declaración con tamaño fijo
```

7. Arrays unidimensionales de tipo long:

```
long myArray[] = {1000000, 2000000, 3000000, 4000000, 5000000}; // Inicialización explícita
```

```
long anotherArray[5]; // Declaración con tamaño fijo
```

8. Arrays unidimensionales de tipo unsigned int:

```
unsigned int myArray[] = {1, 2, 3, 4, 5}; // Inicialización explícita unsigned
```

```
int anotherArray[5]; // Declaración con tamaño fijo
```

9. Arrays unidimensionales de tipo long long:

```
long long myArray[] = {1000000000LL, 2000000000LL, 3000000000LL, 4000000000LL, 5000000000LL}; // Inicialización explícita
```

```
long long anotherArray[5]; // Declaración con tamaño fijo
```

10. Arrays unidimensionales de tipo unsigned long long:

```
unsigned long long myArray[] = {1000000000ULL, 2000000000ULL, 3000000000ULL, 4000000000ULL, 5000000000ULL};  
// Inicialización explícita
```

```
unsigned long long anotherArray[5]; // Declaración con tamaño fijo
```

Ejercicios prácticos con Arrays en C++.

1. Declaración e inicialización de un vector.

Escriba un programa que declare e inicialice un vector de tamaño 5 con valores enteros. El programa debe imprimir en pantalla los valores almacenados en el vector.

2. Obtener valor de casilla específica.

Escriba un programa que declare e inicialice un vector de tamaño 10 con valores enteros. Luego, pida al usuario que ingrese un número entero entre 0 y 9 (inclusive) y el programa debe imprimir en pantalla el valor almacenado en la casilla correspondiente del vector.

3. Recorrido de un vector.

Escriba un programa que declare e inicialice un vector de tamaño 7 con valores enteros. Luego, el programa debe recorrer el vector e imprimir en pantalla los valores almacenados en él. Utilice un ciclo for para recorrer el vector.

4. Suma de elementos de un vector.

Escriba un programa que declare e inicialice un vector de tamaño 8 con valores enteros. El programa debe calcular la suma de todos los valores almacenados en el vector e imprimir los valores en pantalla, también el resultado de la suma.

5. Escriba un programa que pida al usuario que ingrese el tamaño de un vector de enteros. Luego, el programa debe pedir al usuario que ingrese los valores para inicializar el vector. El programa debe imprimir en pantalla el vector inicializado por el usuario y calcular la suma de todos los valores almacenados en el vector. Debe asegurarse de que el programa funcione correctamente para cualquier tamaño de vector ingresado por el usuario.

Resumen de arrays unidimensionales o vectores en C++:

En resumen, los arrays unidimensionales o vectores son una estructura de datos muy útil en C++ para almacenar una colección de elementos del mismo tipo.

En esta clase hemos visto cómo declarar e inicializar vectores en C++, cómo acceder a los elementos de un vector y cómo recorrer un vector utilizando un ciclo for. También hemos visto algunos ejemplos prácticos de cómo utilizar vectores para resolver problemas comunes en programación.

Encuesta para evaluar la clase de arrays o vectores en C++:

1. ¿Qué te pareció la clase sobre vectores en C++?

a. Excelente b. Buena c. Regular d. Mala

2. ¿Entendiste bien cómo declarar e inicializar vectores en C++?

a. Sí, lo entendí completamente b. Más o menos, pero todavía tengo algunas dudas c. No, necesito más explicaciones

3. ¿Qué te pareció la explicación sobre cómo acceder a los elementos de un vector?

a. Muy clara y fácil de entender b. Algo confusa, pero finalmente lo entendí c. No la entendí bien

4. ¿Te resultó útil la sección de ejemplos prácticos?

a. Sí, me resultaron muy útiles b. Algunos ejemplos fueron útiles, pero otros no tanto c. No me resultaron útiles

5. ¿Te gustaría ver más temas relacionados con estructuras de datos en C++ en próximas clases?

a. Sí, me gustaría ver más temas sobre estructuras de datos b. No me interesa mucho ese tema c. Me gustaría ver otros temas en próximas clases

6. ¿Qué parte de la clase te resultó más difícil o confusa?

7. ¿Qué otra información te gustaría haber aprendido en esta clase?

8. ¿Tienes alguna sugerencia o comentario para mejorar esta clase?

Ejercicios propuestos con Arrays en C++.

PARTE TEORICA

BUSCA EN INTERNET Y CONTESTA LOS SIGUIENTES PLANTEAMIENTOS:

1. ¿Qué es un array o arreglo C++?

2. ¿Qué tipo de datos se puede almacenar en un array en C++?

3. ¿Cómo se define un array unidimensional, arreglo o vector en C++?

4. ¿Se puede definir un array y darle valor inicial al mismo tiempo en C++?
5. ¿Después de definir un array se puede inicializar en C++?
6. ¿En un array se puede almacenar distintos tipos de datos? ¿Porqué?
7. ¿Cómo se identifica un elemento de un array en C++?
8. ¿Cómo se accede a un elemento de un array en C++?
9. ¿Cuál es el valor inicial del índice en un array en C++?
10. ¿Cuál es el valor final del índice en un array en C++?
11. ¿Cuál es la función para saber la longitud de un array en C++?
12. ¿Cómo se puede recorrer un array unidimensional en C++?

PARTE PRACTICA

REALIZA LOS SIGUIENTES PROGRAMAS EN C++:

1. Diseñar un programa que permita mostrar por pantalla los siguientes elementos Enteros:

X ---->

23	45	65	76	87	89	32	11	22	33	44	55	66	77	88	99	5
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

A través de un arreglo Unidimensional inicializado con dichos elementos.

2. Diseñar un programa que permita declarar un arreglo Unidimensional con los siguientes Elementos:

x→	22	1	34	85	13	12	3	5	8	11
	0	1	2	3	4	5	6	7	8	9

Mostrar por pantalla aquellos Elementos que son pares.

3. Diseñar un programa que permita generar aleatoriamente números enteros entre un rango de 1 a 100, almacenar dichos números en un arreglo unidimensional de tamaño 20, sabiendo que se tendrá que mostrar por pantalla aquellos que solamente sean pares y múltiplos de 5.
4. Diseñar un programa que permita ingresar por teclado 10 números enteros en un arreglo unidimensional y mostrar dichos elementos por pantalla.
5. Diseñar pprograma que permita asignar elementos de tipo entero a un Arreglo Unidimensional de tamaño 11, calcular el mayor y el menor.
6. Escribir un programa que pida 10 números enteros por teclado y que imprima por pantalla:
- Cuántos de esos números son pares.
- Cuál es el valor del número par máximo.
- Cuál es el valor del número par mínimo.
7. Escribir un programa que lea un vector de 10 elementos. Deberá imprimir el mismo vector por pantalla, pero invertido. Ejemplo: dado el vector 1 2 3 4 5 6 7 8 9 10 el programa debería imprimir 10 9 8 7 6 5 4 3 2 1.
8. Escribir un programa que lea 10 números por teclado. Luego lea dos más e indique si éstos están entre los anteriores.

9. Implementar un programa que:

Declare un vector de 15 elementos.

Almacene un 0 en cada uno de los elementos.

Muestre por pantalla todos los elementos.

10. Implementar un programa similar al del ejercicio anterior, pero en este caso que almacene un 1 en el primer elemento, un 2 en el segundo, un 3 en el tercero, etc.

11. Implementar un programa similar al del ejercicio anterior, pero en este caso que almacene el número 15 en el primer elemento, el 14 en el segundo, el 13 en el tercero, etc.

12. Implementar un programa que pida la introducción de 10 números y los almacene en un vector, a continuación, calcule el promedio de los valores introducidos, luego reste a cada elemento del vector el promedio obtenido y, finalmente, muestre el vector resultante por pantalla.

13. Implementar un programa que solicite la introducción de dos vectores de tamaño N y almacene en un tercer vector la suma (elemento a elemento) de los dos anteriores. Finalmente deberá mostrarse el vector resultante por pantalla.

14. Implementar un programa que solicite la introducción de un vector de tamaño 10, a continuación, solicite un nuevo valor, y compruebe si dicho valor se encuentra o no en el vector.

15. Implementar un programa que solicite la introducción de un vector de tamaño N, invierta su contenido (intercambie el primer elemento con el último, el segundo con el penúltimo, etc.) y lo muestre por pantalla.

Contenido

Arrays, arreglos o vectores en C++	2
Presentación del tema.	2
Objetivos de la clase:.....	2
Declarar e inicializar un array unidimensional o vector en C++.	2
Ejercicios prácticos con Arrays en C++.	4
Resumen de arrays unidimensionales o vectores en C++:	5
Encuesta para evaluar la clase de arrays o vectores en C++:.....	5
Ejercicios propuestos con Arrays en C++.	6
PARTE TEORICA.....	6
BUSCA EN INTERNET Y CONTESTA LOS SIGUIENTES PLANTEAMIENTOS:.....	6
PARTE PRACTICA	7
REALIZA LOS SIGUIENTES PROGRAMAS EN C++:	7
Contenido.....	10